Matière : Dev. App. Rép.
Niveau d'études : L3 Info

Enseignante *: Malek Ltaief*
Année universitaire : 2026-2027

TP1 : Programmation réseau avec les sockets TCP en JAVA

Objectifs du TP

À l'issue de ce TP, vous serez capable de :

— Distinguer le rôle d'un socket d'écoute et d'un socket d'échange ;

— Concevoir et implémenter un client et un serveur TCP en Java, en respectant les étapes de création, connexion et fermeture des sockets ;

— Échanger des données de type entier, chaînes de caractères et objet sérialisable via les flux d'échanges ;

— Déployer et tester l'application client-serveur, en local, puis entre deux machines distinctes reliées par un réseau.

Contexte

Pattes Solidaires est le club de bénévoles de l'ISIMM, actuellement en cours de création, qui s'occupera des chats et chiens errants présents dans la cour de l'institut, le jardin et le parking. Dans le cadre de la préparation de ce club, une application mobile sera développée : les bénévoles s'en serviront sur le terrain, en tant que **client** qui se connecte à un **serveur central** du club pour :

— estimer une ration de nourriture adaptée à un animal repéré ;

— consulter une information utile sur une zone ou un thème donné ;

— enregistrer la fiche d'un animal repéré ou pris en charge.

Exercice 1 : Échange d'entiers

Un bénévole repère un chat ou un chien errant dans la cour, le jardin ou le parking et souhaite lui laisser une ration de nourriture adaptée. Il estime le poids de l'animal (en kg) et l'envoie au serveur central du club, qui calcule et renvoie la quantité de croquettes à distribuer, en grammes.

Travail demandé

1. Développer une application **client/serveur** utilisant les **sockets TCP**. Le client (bénévole) lit au clavier un nombre entier représentant le poids estimé de l'animal en kg (`int`) et l'envoie au serveur via le flux de sortie (`OutputStream`).
2. Le serveur reçoit cet entier via le flux d'entrée (`InputStream`) et calcule la ration recommandée selon la formule simplifiée :
$$\text{ration (g)} = \text{poids_kg} \times 25$$
3. Le serveur applique une règle de garde-fou : si le poids saisi est inférieur ou égal à 0 ou supérieur à 90 (valeur jugée irréaliste pour un animal errant), il renvoie la valeur `0` accompagnée, côté client, du message `"Poids invalide, merci de vérifier votre estimation."`
4. Le client affiche : `"Ration recommandée : {ration} g de croquettes."`

Exercice 2 : Échange de chaînes de caractères

Avant d'intervenir, un bénévole veut consulter une information utile sur le terrain. Il envoie un mot-clé (nom d'une zone ou d'un thème) au serveur central, qui lui répond par un conseil ou une information à jour.

Travail demandé

1. Développer une application **client/serveur** utilisant les **sockets TCP**. Le client envoie une chaîne de caractères (`String`) représentant le mot-clé recherché par le bénévole.
2. Le serveur reçoit ce mot-clé et retourne une réponse selon les règles suivantes (recherche insensible à la casse) :
   - `"COUR"` → `"Cour de l'institut : point de nourrissage principal, prévoir de l'eau fraîche."`
   - `"JARDIN"` → `"Jardin : abri installé sous les buissons, rester discret."`
   - `"PARKING"` → `"Parking : présence récente de chatons, ne pas s'approcher de la mère."`
   - `"VACCIN"` → `"Le prochain passage du vétérinaire bénévole est prévu vendredi."`
   - `"FIN"` ou `"QUITTER"` → le serveur envoie `"Merci pour votre engagement, à bientôt !"` puis ferme la connexion
   - toute autre valeur → `"Information non reconnue, contactez le responsable du club."`
3. Le client affiche la réponse reçue et peut envoyer un nouveau mot-clé tant que la connexion n'est pas fermée par le serveur.

1 Exercice 3 : Échange d'objets

Lorsqu'un bénévole repère ou prend en charge un animal, il doit enregistrer une **fiche animal** dans le registre du club. On souhaite simuler cet enregistrement à l'aide de sockets TCP, en échangeant cette fois un **objet Java sérialisé** plutôt qu'un simple entier ou une simple chaîne.

Travail demandé

a) Définir la classe `FicheAnimal` : Créer une classe `FicheAnimal` qui implémente `Serializable` et qui contient les attributs suivants :

```java
import java.io.*;
import java.time.LocalDate;

class FicheAnimal implements Serializable {
    private String espece;       // "CHAT" ou "CHIEN"
    private String zone;         // "COUR", "JARDIN" ou "PARKING"
    private String etatSante;    // ex : "Blessé", "En bonne santé", "A vacciner"
    private LocalDate dateSignalement;
    private static int compteurFiches = 0;

    public FicheAnimal(String espece, String zone, String etatSante) {
        this.espece = espece;
        this.zone = zone;
        this.etatSante = etatSante;
        this.dateSignalement = LocalDate.now();
        compteurFiches++;
    }

    public boolean estFicheValide() {
```

```
        boolean especeValide = espece.equals("CHAT") || espece.equals("CHIEN");
        boolean zoneValide = zone.equals("COUR") || zone.equals("JARDIN") || zone.
            equals("PARKING");
        return especeValide && zoneValide;
    }

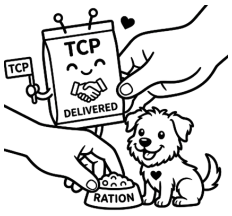
    public String getEspece() { return espece; }
    public String getZone() { return zone; }
    public String getEtatSante() { return etatSante; }
    public LocalDate getDateSignalement() { return dateSignalement; }
    public static int getCompteurFiches() { return compteurFiches; }
}
```

b) Le client (bénévole) crée un objet `FicheAnimal` à partir des informations saisies au clavier (espèce, zone, état de santé) et l'envoie au serveur via un `ObjectOutputStream`.

c) Le serveur (registre central du club) reçoit l'objet via un `ObjectInputStream`, appelle la méthode `estFicheValide()` et :

- si la fiche est valide, il l'ajoute au registre du club (par exemple une simple liste en mémoire) et renvoie au client un objet `AccuseReception` (à définir, contenant un statut `"OK"` et le numéro d'ordre de la fiche) ;
- sinon, il renvoie un `AccuseReception` avec le statut `"REJETE"` et un message précisant l'erreur (espèce ou zone non reconnue).

Congratulations, wonderful heroes !
You have accomplished your mission, and countless cats and dogs thank you !



TCP never lets a single packet get lost along the way !
A kind heart never lets a stray animal go without its ration !